Highlights

Design and Evaluation of an Autonomous Cyber Defence Agent using DRL and an Augmented LLM

Johannes Loevenich,Erik Adler,Tobias Hürten,Roberto Rigolin F. Lopes

- Design of a hybrid AI architecture integrating an Autonomous Cyber Defense (ACD) agent using Deep Reinforcement Learning (DRL) with a chatbot using a Retrieval Augmented Generation (RAG) architecture and cybersecurity knowledge graphs. Our ACD agent has an action space composed of 14 actions to monitor, analysis, deploy a decoy replacing a real service, remove malicious processes, and restore the entire systems from a backup.
- Design and implementation of three knowledge graphs that combine information about the network infrastructure and open source Cyber Threat Intelligence (CTI) reports. This includes an architecture for automatically mapping Common Vulnerabilities and Exposures (CVE) to the set of offensive tactics and techniques defined in the MITRE ATT&CK framework, using generative models based on Bidirectional Encoder Representations from Transformers (BERT) and Generative Pre-trained Transformer (GPT). Our solution was evaluated in a comparative performance analysis using three BERT-based models (CyBERT, SecBERT and SecRoBERTa) and two versions of GPT (GPT-4 and GPT-4o).
- Comparative evaluation of an ACD agent acting on the cybersecurity knowledge graph against 11 DRL algorithms, Proximal Policy Optimization (PPO) (baseline for the comparison), hierarchical PPO and ensemble DRL, in a gym environment hosting red agents using two attack strategies. Experimental results indicate that our solution using PPO and cybersecurity knowledge graphs outperforms the other DRL algorithms.
- Implementation and performance evaluation of a chatbot (ReAct+RAG) using a pre-trained Large Language Model (LLM) to interface with human cybersecurity experts. Our solution using ReAct and RAG was evaluated using both generation (relevancy, faithfulness and semantic similarity) and retrieval (context precision, context recall, and F1 score) metrics in a comparative analysis against a zero-shot implementation (pre-trained LLM without augmentation).

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

Design and Evaluation of an Autonomous Cyber Defence Agent using DRL and an Augmented LLM

Johannes Loevenich[a,b,*] (Researcher), Erik Adler[a,c] (Researcher), Tobias Hürten[a,d] (Researcher) and Roberto Rigolin F. Lopes[a] (Researcher)

[a]*Secure Communications & Information (SIX), Thales Deutschland GmbH, Thalesplatz 1, Ditzingen, 71254, BW, Germany*
[b]*Department of Mathematics/Computer Science, University of Osnabrück, Neuer Graben 29, Osnabrück, 49074, NI, Germany*
[c]*Department of Informatics, Karlsruhe Institute of Technology (KIT), Kaiserstraße 12, Karlsruhe, 76131, BW, Germany*
[d]*Department of Computer Science IV, University of Bonn, Regina-Pacis-Weg 3, Bonn, 53113, NW, Germany*

ARTICLE INFO

Keywords:
Autonomous Cyber Defense
Autonomous Cyber Operation Gym
Cybersecurity Knowledge Graph
Deep Reinforcement Learning
Large Language Model
Proximal Policy Optimization
Retrieval-Augmented Generation

ABSTRACT

In this paper, we design and evaluate an Autonomous Cyber Defense (ACD) agent to monitor and act within critical network segments connected to untrusted infrastructure hosting active adversaries. We assume that modern network segments use software-defined controllers with the means to host ACD agents and other cybersecurity tools that implement hybrid AI models. Our agent uses a hybrid AI architecture that integrates deep reinforcement learning (DRL), augmented Large Language Models (LLMs), and rule-based systems. This architecture is implemented in software-defined network controllers, enabling automated defensive actions such as monitoring, analysis, decoy deployment, service removal, and recovery. A core contribution of our work is the construction of three cybersecurity knowledge graphs that organize and map data from network logs, open source Cyber Threat Intelligence (CTI) reports, and vulnerability frameworks. These graphs enable automatic mapping of Common Vulnerabilities and Exposures (CVEs) to offensive tactics and techniques defined in the MITRE ATT&CK framework, using Bidirectional Encoder Representations from Transformers (BERT) and Generative Pre-trained Transformer (GPT) models. Our experimental evaluation of the knowledge graphs shows that BERT-based models achieve better performance, with precision (83.02%), recall (75.92%), and macro F1 scores (58.70%) significantly outperforming GPT models. The ACD agent was evaluated in a Cyber Operations Research Gym against 11 DRL models, including Proximal Policy Optimization (PPO) and Hierarchical PPO, under two different attacker strategies. Results show that the ACD agent outperformed baseline implementations, with its DRL models effectively mitigating attacks and recovering compromised systems. In addition, we implemented and evaluated a chatbot using Retrieval-Augmented Generation (RAG) and a prompting agent augmented with the CTI reports represented in the cybersecurity knowledge graphs. The chatbot achieved high scores on generation metrics such as relevance (0.85), fidelity (0.83), and semantic similarity (0.88), as well as retrieval metrics such as contextual precision (0.91). The experimental results suggest that the integration of hybrid AI systems with knowledge graphs can enable the automation and improve the precision of cyber defense operations, and also provide a robust interface for cybersecurity experts to interpret and respond to advanced cybersecurity threats.

1. Introduction

Recent developments in Deep Reinforcement Learning (DRL), Large Language Models (LLMs), and Software Defined Networking (SDN) have the potential to enhance the effectiveness of autonomous cyber defense solutions to detect and mitigate cyber attacks in critical networks connected to untrusted network infrastructure (Kiely et al., 2023; Loevenich et al., 2024b; Lopes et al., 2023; Velazquez et al., 2023a,b). Therefore, this investigation assumes that the control plane of critical network segments has the communication and computational resources to host Autonomous Cyber Defense (ACD) agents automating the enforcement of defensive tactics and techniques using hybrid Artificial Intelligence (AI) models. The goal is to design, implement, train and test robust autonomous agents that can efficiently monitor and defend network segments supporting information exchange between two or more nations in a coalition using the set of standards for Protected Core Networking (PCN) developed by North Atlantic Treaty Organization (NATO).

ACD agents can implement hybrid AI models combining background knowledge from threat intelligence sources such as MITRE (MITRE, 2024a) or NIST (Hamid et al., 2012) with deep learning models to detect known vulnerabilities in large datasets with system's logs and data streams (Loevenich et al., 2024a; Piplai et al., 2023). For example, neuro-symbolic models integrate neural and symbolic AI architectures to address the weaknesses of each other, providing robust hybrid AI models capable of reasoning, learning, and cognitive modeling (Piplai et al., 2023). Neural networks excel at pattern recognition, but they can be difficult to interpret and explain. Symbolic AI uses logic and reasoning to solve problems, but they can be inefficient and struggle to handle complex problems. Because these problems may demand a combinatorial explosion of rules, which

*Corresponding author
johannes.loevenich@thalesgroup.com (J. Loevenich); erik.adler@thalesgroup.com (E. Adler); tobias.huerten@thalesgroup.com (T. Hürten); roberto.rigolin@thalesgroup.com (R.R.F. Lopes)
ORCID(s): 0000-0002-8149-1600 (J. Loevenich); 0009-0003-2127-6399 (E. Adler); 0009-0005-2121-0810 (T. Hürten); 0000-0002-0114-5610 (R.R.F. Lopes)

Preprint not peer reviewed
This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

are costly to maintain and update. As a result, hybrid AI architectures/systems combine the strengths of both to create systems that are simultaneously efficient and interpretable.

The present investigation introduces a ACD agent architecture using a DRL model, a chatbot, and three knowledge graphs generated using monitored system logs, MITRE's ATT&CK framework containing tactics and techniques, and Common Vulnerabilities and Exposuress (CVEs)/Common Weakness Enumerations (CWEs) from the National Institute of Standards and Technology (NIST). To its end, we propose a methodology to build three knowledge graphs, a Resource Description Framework (RDF) graph representing the monitored system logs and architecture from the network, a labeled property graph representing the knowledge gathered from open source Cyber Threat Intelligence (CTI) reports, and a third graph mapping the RDF graph to the labeled property graph, thus linking CVEs to system logs that are continuously monitored by the system. The ACD agent implements a set of 14 actions to monitor, analyze, remove and restore services/nodes, as well as to employ decoy services. The agent is trained within the Cyber Operations Research Gym (CybORG) environment against red agents using two different attack strategies to perform offensive actions such as discovery, exploit, privilege escalation and impact (CAGE, 2022). The chatbot uses a pre-trained LLM (GPT-3.5-turbo-16k), which is augmented with cybersecurity knowledge graphs, as an external data source, using ReAct ("reasoning and acting") prompting (Yao et al., 2023), together with Retrieval Augmented Generation (RAG) techniques (Lewis et al., 2020). As a result, cybersecurity experts can write prompts asking for explanation about offensive/defensive tactics and techniques related to a set of events in the network environment.

In summary, the contributions of this paper are:

1. Design of a hybrid AI architecture that integrates an ACD agent using DRL with a chatbot using a pre-trained LLM augmented by cybersecurity knowledge graphs. Our architecture defines the main functional blocks in a Automated Cyber Operation (ACO) gym environment hosting blue and red agents.

2. Design and implementation of Type I, Type II and Type III knowledge graphs that combines multi-layer logs from the network infrastructure and open-source CTI reports. This includes an architecture for automatically mapping CVEs to a set of offensive tactics and techniques from the MITRE ATT&CK framework using Bidirectional Encoder Representations from Transformers (BERT).

3. Comparative evaluation of an ACD agent acting on the knowledge graph against baseline hierarchical and ensemble DRL implementations in a gym environment that hosts two types of red agents.

4. Implementation and performance evaluation of a chatbot (ReAct+RAG) using a pre-trained LLM to interface with human cybersecurity experts. We performed a quantitative evaluation using both generation and retrieval metrics.

The rest of this paper is organized as follows. Section 2 discusses the motivation for the present investigation and related works on using hybrid AI models to automate defensive cybersecurity tasks. Section 3 describes the functional entities within the gym environment, including the autonomous agent architecture and the architecture to generate cybersecurity knowledge graphs. Section 4 describes the network topology, red agents, and ACD agent deployed in the gym environment. Section 5 presents a comparative performance analysis of the ACD agent and discusses generation/retrieval results from the chatbot. Finally, Section 6 concludes the paper and lists future work.

2. Background

This section discusses the motivation for automating the enforcement of defensive cybersecurity measures using hybrid AI models within agents (Section 2.1) and reviews related literature on training/testing ACD agents, mapping known vulnerabilities to know offensive tactics/techniques (Section 2.2.2), and augmenting pre-trained LLMs (Section 2.2).

2.1. Motivation

Recent developments in AI models and the emerging agentic paradigm suggest that it is possible to train and test a set of autonomous agents with complementary expertise collaborating to defend critical networks and systems as a team. As a result, the blue team comprises cooperation between human cybersecurity experts and a set of ACD agents. This approach is motivated by human soldiers with different expertise who are trained by the military to work together as a team during operations in hostile environments. For example, a squad can have soldiers with distinct know-how and equipment actuate as infantry, snipers, demolition / bomb disposal experts, drone operators, tactical electronic warfare specialists, and so on. In an analogy between humans and machines, we start with the hypothesis that it is possible to train a set of specialized agents collaborating in a team to defend critical networks. The ultimate goal is to train and test robust and secure agents (Loevenich et al., 2024c).

For example, Fig. 1 plots a hierarchy of exemplary models/agents with intelligence/adaptability as a function of computation/communication resources. In this figure, the y-axis ranges from weak intelligence (for a particular domain) towards Artificial General Intelligence (AGI), whereas the x-axis ranges from low use of resources to high availability of resources, such as computing power, storage, connectivity and energy supply. The current race to improve the performance of generative AI models, e.g. LLMs and foundation models, may significantly impact the way we design, implement, and improve robust/safe AI models for critical systems in the long term future. The scientific community is leveraging recent developments to increase collaboration among autonomous agents in a team,

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

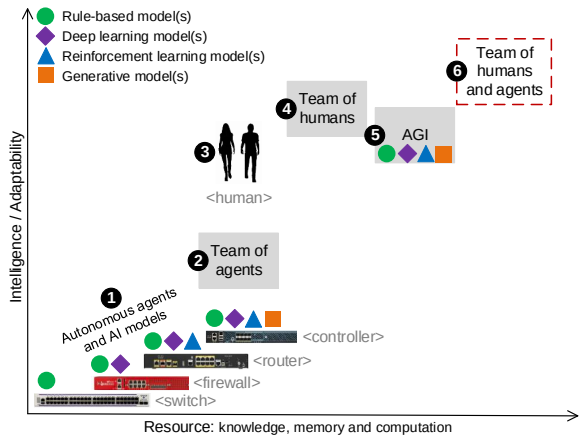


Figure 1: Intelligence and adaptability as a function of resources for autonomous agents and teams of humans/agents (Loevenich et al., 2024b).

as well as with teams of human cybersecurity experts. The following definitions are used in Fig. 1 and throughout this paper.

(1) *Autonomous agent*: A software application implementing hybrid AI models (rule-based, deep learning, reinforcement learning or generative AI models) with the means to monitor/actuate in the environment (network, operating system and services) to detect or mitigate cyberattacks.

(2) *Team of agents*: Set of autonomous agents collaborating to achieve individual and shared goals, such as defending critical infrastructure networks and systems.

(3) *Human expert*: A cybersecurity expert trained to monitor, maintain and defend networks/systems/services using established tools/techniques.

(4) *Team of humans*: Set of cybersecurity experts and their tools collaborating in a team to protect a particular network segment and systems/services.

(5) Artificial General Intelligence (AGI): Abstract representation of future systems with general intelligence equal or superior to a human being and strong generalization capabilities (Goertzel, 2014).

(6) *Team of humans and agents*: Set of cybersecurity experts and ACD agents collaborating in a team using natural language to solve complex problems.

2.2. Related Work

This section reviews recent investigations also developing ACD agents (Section 2.2.1), cybersecurity knowledge bases (Section 2.2.2), and defining metrics to evaluate augmented LLMs (Section 2.2.3).

2.2.1. Reinforcement Learning for ACD

Autonomous Cyber Defense (ACD) systems are designed to protect networks and information systems against constantly evolving threats (Kott et al., 2019). A key challenge in this area is the need for systems capable of operating in highly dynamic and adversarial environments, often requiring real-time decision making. CybORG, an AI gym for cyber operations, provides a framework for training DRL agents in simulated cyber environments (Standen et al., 2021). This tool enables the development and evaluation of ACD agents by simulating both red (attacker) and blue (defender) team operations, allowing different AI models to be tested in realistic settings. Similar environments, such as NASim (Becker et al., 2024), have also been proposed for training autonomous penetration testing agents, highlighting the growing interest in simulation-based environments for cybersecurity research.

Most automated approaches are model-based, e.g. in Booker and Musman (2020) the authors propose a model-based decision-theoretic approach for automated cyber response that relies on the framework of Partially Observable Markov Decision Processes (POMDPs) to simulate and plan defensive strategies, allowing for real-time decision making under uncertainty. DRL has emerged as a promising approach for handling POMDPs and automating cybersecurity tasks. DRL models can learn to optimize defense strategies by interacting with complex environments, making them suitable for cyber defense applications (Fard et al., 2023).

In Liu et al. (2024), the authors present a DRL-based approach to optimizing mitigation deployment using an extended version of MITRE's ATT&CK framework. By prioritizing critical nodes in a network, their model reduces redundancy in mitigation strategies while maintaining an effective defense against sophisticated cyber threats. In contrast, the investigation in Huang and Zhu (2019) proposed a Bayesian game framework to model the interaction between defenders and Advanced Persistent Threats (APTs) in critical network infrastructure. Their approach uses Bayesian Nash equilibria to anticipate attacker behavior and optimize defense strategies.

The investigation reported in Gao and Wang (2021) explored the use of DRL in Moving Target Defense (MTD) against Distributed Denial of Service (DDoS) attacks. Their adaptive approach dynamically adjusts defense strategies to counter evolving attacker behavior while balancing system performance and security. Another study (Becker et al., 2024) evaluates the use of various Reinforcement Learning (RL) algorithms, including Asynchronous Actor-Critic (A3C) and Q-learning, for autonomous penetration testing. The results show that DRL models outperform traditional penetration testing methods, especially in scenarios that require generalization across different environments. This highlights the potential of DRL to reduce human involvement in penetration testing and other human labor intensive cybersecurity tasks.

In addition, recent research has explored the use of causality in DRL to improve model performance. In Peng et al. (2024), the authors developed a causality-driven hierarchical DRL framework that improves exploration efficiency

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

by exploiting the causal relationships between environmental variables. Similarly, in Zhu et al. (2023), the authors propose a method that mitigates bias in offline DRL models caused by unobserved confounders, further demonstrating the importance of causal reasoning in DRL-based cyber defense. The investigation in Rezende et al. (2020) highlights the importance of causally correct partial models in RL to prevent confounding and ensure accurate policy learning, especially in high-dimensional environments. In Zhu et al. (2023), the authors proposed a deconfounding framework for offline DRL that re-weights observational data to mitigate biases introduced by unobserved confounders. This method ensures unbiased policy learning and improves the reliability of offline DRL models in practical scenarios.

Hybrid AI models, which combine different AI techniques such as deep learning and symbolic AI, are gaining importance in the cybersecurity domain due to their ability to leverage both data-driven learning and rule-based reasoning. The investigation in Piplai et al. (2023) suggests that neuro-symbolic AI, which integrates neural networks with symbolic reasoning, can address some of the limitations of pure neural models, such as lack of interpretability. This is particularly important in cybersecurity, where decisions made by AI models often need to be transparent and explainable to human operators. Recent developments in Multi-Agent Reinforcement Learning (MARL) and causality-driven frameworks have further enhanced the capabilities of ACD systems. For example, in Huang and Zhu (2019) the authors employed Multi-Agent Reinforcement Learning (MARL) in conjunction with Bayesian inference to address the stealthy and deceptive behaviors of APTs. Their results indicate that Bayesian updates and equilibrium solutions significantly improve defender performance.

2.2.2. Mapping CVEs to MITRE ATT&CK

This investigation starts with the hypothesis that ACD agents and cybersecurity experts can benefit from a knowledge base combining CVEs and the set of offensive tactics, techniques and mitigation strategies compiled in the MITRE ATT&CK matrix (MITRE, 2024a). The goal is to provide a critical link between vulnerability management, threat modeling, and mitigating controls. This mapping allows cybersecurity experts and agents to contextualize vulnerabilities by understanding the attacker tactics and techniques associated with a CVE, enabling them to better assess its potential impact and prioritize mitigation efforts. Furthermore, ATT&CK mappings enhance threat models by incorporating real-world attacker behaviors and potential exploit paths. Finally, mapping CVEs to ATT&CK facilitates the selection and implementation of targeted security controls that disrupt specific adversary tactics and techniques, ultimately strengthening the overall cybersecurity posture.

In Kuppa et al. (2021), the authors introduced a multi-label text classification approach for mapping CVEs to ATT&CK techniques and proposed a multi-head joint embedding neural network architecture. They addressed the problem of insufficiently labeled data by using an unsupervised labeling method that assigned labels to 17 MITRE ATT&CK techniques without supervision. Their methodology extracts phrases from threat reports and vulnerability descriptions, and then maps vulnerabilities to technologies based on the cosine distance between the vulnerability description and the technology vector. This unsupervised approach correctly maps a subset of only 17 out of ∼300 offensive techniques, leaving room for significant improvement in terms of both accuracy and coverage.

The investigation reported in Ampel et al. (2021) compared several techniques and their performance for mapping CVEs to MITRE: classical Machine Learning (ML) models (i.e., random forests, Supported Vector Machine (SVM), naive Bayes, and logistic regression), deep learning models (Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), and BiLSTM), pretrained language models (GPT-2, BERT, and A Robustly Optimized BERT Pretraining Approach (RoBERTa)), and self-distillation. Their distillation approach combines relational knowledge from a large pre-trained model and a prior untrained model. As a result, the pre-trained model generalizes better to unseen data than a model without knowledge distillation. This approach yielded encouraging results, with an F1 score of 76.18%, however, the classification was only multiclass, not multilabel, as a CVE was only mapped to a corresponding tactic. This rarely happens in the real world, where a vulnerability is often exploited by multiple tactics.

On the same data set, Grigorescu et al. (2022) also presented a multi-label text classification approach. The difference is that the authors mapped offensive techniques instead of tactics. Ideally, the mapping of techniques is the ultimate goal, however, only a few techniques (31 out of 192) exceeded a minimum occurrence threshold, and several limitations were observed, especially due to the high imbalance in the dataset. Therefore, this study focuses on tactics as an overarching goal. The text descriptions of CVEs were classified using A BERT Model for Scientific Text (SciBERT) and adversarial attacks for the 31 techniques with an F1 score of 47.84%.

In Haddad et al. (2023), the authors mapped CVEs to the MITRE CWE Top 25 Weaknesses by approaching the problem as a ranking problem and aiming to publish a dataset of 4012 manually annotated samples. The main difference between CVEs and CWEs is that the CWE focus on identifying and categorizing specific vulnerabilities or weaknesses, while the MITRE tactics focus on categorizing and describing adversary behaviors and objectives during cyber attacks. Fine-tuned deep learning models such as Sentence BERT (SBERT) and rankT5 provided good semantic understanding for this ranking task. Specific ranking task metrics such as Mean Reciprocal Rank (MRR), Mean Average Precision (MAP), and Normalized Discounted Cumulative Gain (NDCG) were used to evaluate the models. MAP@k/NDCG@k compares the top-k prediction to the ground truth (true positives), so MAP@1 and NDCG@1 actually represent a classification measure. The best model

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

was a fine-tuned SBERT, with a MAP@1 score of 85% and an NDCG@1 score of 84.46%.

2.2.3. Metrics for Evaluating LLMs

Given an initial prompt, LLMs can predict the next token or word by iteratively feeding its own outputs back into the model (Vaswani et al., 2017). Therefore, prompts are essential for interacting with modern LLMs. A prompt is a natural language text that describes the task that an LLM should perform. It can include a range of information, including the instructions or questions directed to the model, as well as additional details such as context, input, or in-context examples. Because the quality of LLM output can empirically vary depending on how prompts are constructed (Wei et al., 2024), various prompt engineering techniques have been developed to improve the quality of generated responses (Schulhoff et al., 2024), i.e., to achieve output that is consistent with the intent of the prompt author.

For example, LLMs can be extended to include external knowledge sources in the query, such as parts of cybersecurity knowledge graphs, using a Retrieval Augmented Generation (RAG) architecture (Lewis et al., 2020). RAG often injects knowledge into pre-trained LLMs more efficiently than unsupervised fine-tuning (Ovadia et al., 2023), which can be costly and time consuming. The overall goal in RAG is to establish a systematic way of performing Question & Answer (Q&A) to automate prompt engineering (generation and retrieval) and to avoid hallucinations and knowledge cutoffs (Rawte et al., 2023; Zhao et al., 2023). This method builds a reliable knowledge base that supports verification, strengthens defensive strategies, and advances AI-driven capabilities that automate cybersecurity techniques.

Prompts are also used to define the context and assign the LLMs to particular roles (e.g. *"from now on you are a cybersecurity expert who gives detailed answers"*). Observation-based agents, such as ReAct (Yao et al., 2023), use the inherent decision-making capabilities of LLMs to dynamically select and employ external tools within interactive environments based on the observation feedback (Yang et al., 2023). Designed for problem-solving, these agents operate in a structured feedback loop, following the ReAct framework to iteratively generate thoughts, perform actions, and process observations. Actions typically involve interacting with external environments via well-defined interfaces, known as tools, which provide structured outputs back to the agent. This cyclical process of thinking, acting, and receiving observations, is captured within the prompt, allowing agents to retain a form of memory for past states, actions, and insights. Some agents extend this model to interact with more complex or toy environments, further enriching their problem-solving capabilities (Yang et al., 2023).

One approach to evaluating the output of LLMs is to use *n-gram-based* metrics, with ground truths as references. Recall-Oriented Understudy for Gisting Evaluation (ROUGE) (Lin, 2004), in particular ROUGE-1, evaluates the overlap of unigrams between candidate and reference texts, focusing on recall to measure how much of the reference content is captured by the candidate. Higher ROUGE-1 scores, ranging from 0 to 1, indicate a closer content match. Other ROUGE variants, such as ROUGE-2 and ROUGE-L, and Bi-Lingual Evaluation Understudy (BLEU) (Papineni et al., 2002), a metric that balances precision and length for translation quality, are also commonly used in text evaluation. However, these metrics have limitations, especially in generative Q&A models. While ROUGE is effective in distinguishing between strategies, it struggles to accurately evaluate generative Q&A models on its own (Banerjee et al., 2023) because it relies on ground truth labels that may be inaccurate or inconsistent. Word-based metrics often misinterpret synonyms or interchangeable words as errors, even when they match the intended meaning of the reference text. Moreover, these techniques fail to capture the complexities of multi-hop reasoning in Q&A, such as semantics, faithfulness, and responsiveness (Adlakha et al., 2023; Mavi et al., 2022).

In evaluating RAG systems (Lewis et al., 2020), it is important to consider both the *retrieval* of known information and the *generation* of responses to prompts. The retrieval process establishes context, while the generation process, handled by the pre-trained LLM, uses this context to generate responses. Thus, evaluating a RAG architecture requires separate evaluations of the retrieval and generation components, along with a combined analysis of their overall performance. One effort is Ragas, an evaluation framework designed specifically for RAG (Es et al., 2024; Ragas, 2024). For example, three metrics designed to quantify generation quality are *Faithfulness*, *Relevance*, and *Semantic Similarity*.

The *Faithfulness* metric measures the factual accuracy of a generated response relative to a given context. It generates a proportion of the claims in the answer that are verifiable against the context, and then matches them against the statement derived from the LLM output. The *Faithfulness* score is the ratio of the number of claims inferred from the context to the total number of claims in the generated answer. The *Relevance* metric evaluates how closely the generated response relates to the prompt by calculating the mean cosine similarity between the original question and multiple paraphrased variants of it using an LLM. The *Semantic Similarity* is computed by vectorizing both the ground truth and the generated answers using the specified embedding model, and then computing the cosine similarity between the resulting vectors.

Finally, quantitative metrics such as *Context Precision* and *Context Recall* are used to evaluate the retrieval performance of augmented LLMs implementing RAG architectures. Specifically, context precision measures the relevance of the retrieved context to the question posed, while context recall measures how well the retrieved context matches the information needed to derive the correct answer. Both metrics are evaluated by an LLM on a scale from 0 to 1, following the guidelines of the Ragas framework.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

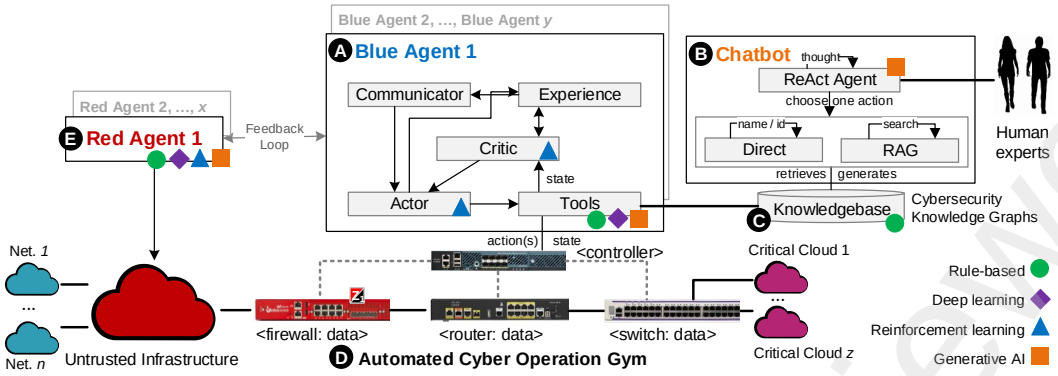


Figure 2: Architecture: ACD agent (A), chatbot (B), knowledgebase (C), gym environment (D) and red agents (E).

3. Design

This section introduces the ACD agent architecture (Section 3.1), defines the model for generating cybersecurity knowledge graphs (Section 3.2), and presents a RAG architecture for the chatbot using a pre-trained LLM to create the interface with cybersecurity experts (Section 3.3).

3.1. Architecture: ACD Agent & Chatbot

Our ACD agent combines rule-based and Deep Reinforcement Learning (DRL) models (A) to detect and mitigate cyberattacks, while using generative AI models for the chatbot (B) with a pre-trained LLM augmented with cybersecurity knowledge graphs (C), as shown in Fig. 2. This figure sketches the overall architecture including the network environment simulated by an ACO gym (D) and the red agents implementing different attack strategies (E). The main functional blocks within (A), (B) and (C) are defined as follows.

- *Actor*: Decentralized agent network that implements the decision making of each blue agent. It is used to map observations to high-level actions, each of which is mapped to a set of rules for executing low-level actions using *Tools* within the network.
- *Tools*: System monitoring and analysis tools to detect malware, tools to install decoy services on the network, and tools to restore or recover operational/enterprise services. For example, Table 1 lists the actions space of our ACD agent implemented by a set of tools.
- *Critic*: A value neural network that represents a learnable approximation of the value function. It monitors real-time execution by evaluating actions and providing continuous feedback to the experience layer. It allows the agent to seamlessly switch between planning and acting based on live updates.
- *Experience*: Recommends actions to manage the situation based on the context provided by the *Critic*. It also updates the Type I - Type III knowledge graphs within the *Knowledgebase* to incorporate new information gathered while defending the environment.
- *Communicator*: Receives tasks and current state data, requests action plans from the *Experience* component, and coordinates the execution of those actions, including any necessary rescheduling due to changes or unexpected outcomes in the environment.
- *Chatbot*: Acts as an interface between the human experts and the tools of the blue agent, keeping the experts informed about the status of the system, the selected tools for mitigating cyber attacks, and updates to the Type I - Type III knowledge graphs within the *Knowledgebase*. It ensures that the model remains explainable, rather than becoming a black box.
- *Knowledgebase*: A database containing the Type I - Type III knowledge graphs using GraphDB and Neo4j, representing CTI and network infrastructures information. It contains information about known threats, mitigation strategies, and other context critical to incident response. More details can be found in Section 5.3.1.
- *Red Agents*: A set of autonomous or semi-autonomous red agents trained to perform various cyber attacks, reconnaissance, and exploitation tactics. For example, Table 2 lists five high-level actions (system discovery, service discovery, exploit, privilege escalation, and impact) implemented by red agents using two attack strategies (more in Section 4.2).

3.2. Cybersecurity Knowledge Graph (CSKG)

The application of knowledge graphs in the cybersecurity domain can be used to organize, manage, and process large amounts of information in cyberspace, such as through ontology-based knowledge representation, which can completely and accurately represent the complex knowledge of heterogeneous systems. When working with cybersecurity knowledge graphs, there are many information security and network process characteristics that need to be stored. The semantics of the captured cybersecurity knowledge varies greatly depending on the graph data model used. We use three types of graphs, namely RDF graphs (Type I) to represent knowledge about the network infrastructure, labeled

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

Table 1
High- and low-level action space for the ACD agent (blue).

High-level Action	Low-level Action	Description	Parameters	Output
Monitor	Query VelociraptorServer	Collects host state information using OpenC2 Language	None	Hosts and services
Analyse	Monitor	Collects host state information using OpenC2 Language	Hostname	Information on files associated with recent alerts
	Density Scout	Calculates density (entropy) of files to find potential Malware		
	SigCheck	Execute SigCheck to scan for not-signed Malware files		
	Query CSKG	Query knowledge graph for potential cyber threats		
Decoy	DecoyApache	Deploy decoy service to appear as Apache Server	Hostname	Alert if decoy service is accessed by red agent
	DecoyFemitter	Deploy decoy service to appear as Femitter FTP Service		
	DecoyHarakaSMPT	Deploy decoy service to appear as Haraka 2.7.0 Service		
	DecoySmss	Deploy decoy service to appear as MS SMB Service		
	DecoySSHD	Deploy decoy service to appear as SSH Service		
	DecoySvchost	Deploy decoy service to appear as SvHost Service		
	DecoyTomcat	Deploy decoy service to appear as Tomcat Server		
Remove	StopProcess	Stops malicious processes	Hostname	Success/Failure
Restore	RestoreFromBackup	Restores a system to a known good state	Hostname	Success/Failure

property graphs (Type II) to represent threat information gathered from open source cybersecurity reports, and a Type III graphs that connects graphs of Type I and Type II.

3.2.1. Definitions

Formally, the Type I RDF graph G_R is a set of RDF triples of the form $(s, p, o) \in (I \cup B) \times I \times (I \cup L \cup B)$, where:

- I is a set of International Resource Identifiers (IRIs) of the form `scheme:[//[user:pwd]host[:port][/]path [?query][#fragment]` or a valid subset of these;
- L represents RDF literals, which are either:
  - Plain, self-denoting literals L_P of the form `"<string>"(@<lang>)`, where `<string>` is a string and `<lang>` is an optional language tag; or
  - Typed literals L_T, of the form `"<string>" <datatype>`, where `<datatype>` is an IRI representing a datatype according to a schema, and `<string>` is an element of the lexical space corresponding to the datatype;
- B is a set of blank nodes, i.e., unique but anonymous resources that are neither IRIs nor RDF literals.

Here, the sets I, L, and B are pairwise disjoint infinite sets. The Type I RDF graph is used to represent a knowledge base about the network infrastructure, and depending on the granularity, the nodes represent either simulated, emulated, or real network infrastructure, network device entities, and their properties, whereas the edges represent the physical and logical links between them.

The second type of graph we use is a labeled property graph, which is of the form $G_{LP} = (V, E, \iota, \lambda, \pi)$, where:

- V is a finite set of vertices or nodes,
- E is a finite set of edges such that V and E are disjoint,
- $\iota : E \to (V \times V)$ is an incidence function that maps each edge in E to a pair of vertices in V,
- $\lambda : (V \cup E) \to L_S$ is a label function that associates an edge or vertex with a set of labels from L_S, and
- $\pi : (V \cup E) \times P \to V_S$ is a property assignment function that assigns a set of values from V_S to each property.

Here, the functions λ and π, are partial functions. Type II is a graph that represents cyber threat intelligence covering open source data from MITRE ATT&CK, NIST and MITRE D3FEND. This graph is a compressed representation of cyber attack methodologies, where tactics, techniques, and mitigation procedures are node types within the graph, and their interrelationships are represented as connecting edges. The graph includes additional node types representing descriptions of common CWEs and CVEs, which are important to describe the broader context of vulnerabilities. These nodes are mapped to the corresponding ATT&CK techniques and D3FEND mitigation procedures providing links within the heterogeneous graph. Finally, the Type III graph is used for risk analysis of the system infrastructure and defined by a mapping between the Type I and Type II graphs.

3.2.2. Architecture

These three types of knowledge graphs are generated by the functional blocks in Fig. 3. The Type I RDF graph is built using GraphDB, which allows us to easily query the normalized network graph constructed from the OpenC2 logs using SPARQL. To construct the Type II graph, we use annotated data generated from the CTI reports using the MITRE Annotation Toolkit (MAT) available in (MITRE, 2024b). The data annotation task is cognitively taxing, time-intensive, and resource-expensive, but training data has the largest impact on the accuracy of our BERT classifier. One other aspect of building training data is the importance of

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

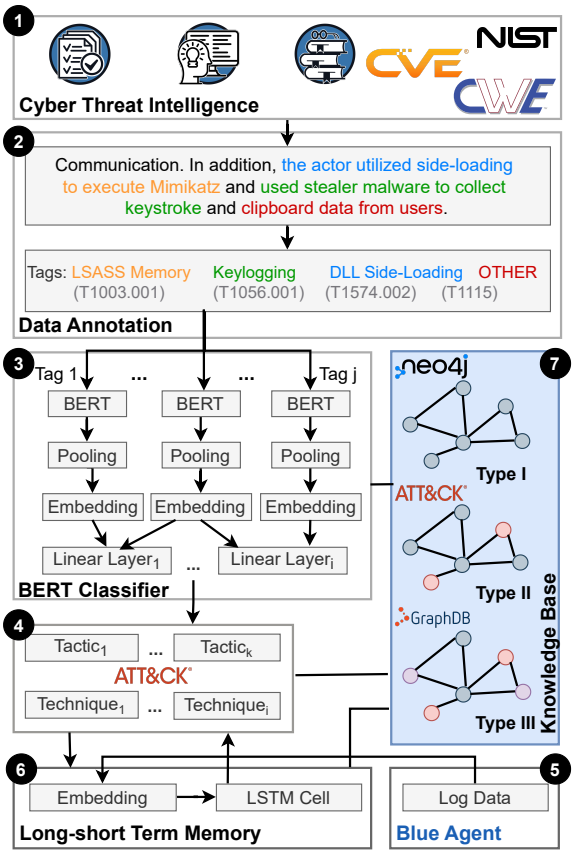


Figure 3: Architecture for creating cybersecurity knowledge graphs of Type I, II, and III.

consistently standardizing the data. We use two approaches to split the large CTI report documents into smaller pieces, as in Fig. 3 (1) and (2).

First, a sentencizer is used to divide the text into sentences and second tags are assigned to highlight sections of text that map to an associated ATT&CK technique. These highlighted sections may be contained entirely within a sentence, or span multiple sentences, or partially overlap two sentences. The document is divided into sentences, which are assigned to the ATT&CK labels that are contained wholly or partially by that particular sentence. This means that some sentences will have no labels, some will have exactly one, and some will have more than one. The result is a multi-label macro dataset containing sentences and their respective labels, as shown in the first example in Listing 1 (lines 1-8).

Another approach is to split the documents into phrase-level tags as illustrated in Fig. 3 (2). In this case, each phrase can be mapped isomorphically to one single ATT&CK technique therefore generating the micro single-label dataset which we use to generate the Type II graph. For example, in Listing 1 the tag Keylogging (green) extracted from the phrase *"used stealer malware to collect keystroke"* will be mapped to technique T1056.001 (lines 13-18).

```
1  {
2     "sentence": "Communication. In addition, the actor
          utilized side-loading to execute Mimikatz and used
          stealer malware to collect keystroke and clipboard
          data from users. ",
3     "labels": [
4        "T1003.001",
5        "T1056.001",
6        "T1574.002",
7        "T1115"
8     ],
9     "doc_title": "NotPetya Technical Analysis
10    A Triple Threat File Encryption MFT
11    Encryption Credential Theft"
12 }
13 {
14    "phrase": "used stealer malware to collect
15              keystroke",
16    "label": "T1056.001",
17    "doc_title": "NotPetya Technical Analysis - A Triple
          Threat: File Encryption, MFT Encryption, Credential
           Theft"
18 },
```

Listing 1: Example sentence and token extracted from the CTI reports using the MITRE Annotation Toolkit.

The BERT classifier uses the architecture shown in Fig. 3 (3). In this figure, the model considers contextual embeddings from the pre-trained BERT encoder, passing through a pooling layer and a linear layer to compute a probability that the input maps to a particular ATT&CK technique or tactic (4). For the loss function, we combine a sigmoid layer with the Binary Cross-Entropy (BCE) loss because the output after the linear layer represents the probabilities that a given CVE points to that particular technique, which needs to be treated independently. Consistently dividing the document allows for the positive samples to be reproducible, as well as, produces negative samples. The model needs to be trained on negative samples so that it can best determine when part of a document does not have an ATT&CK technique.

Finally, we use the monitored log data as input to a LSTM classifier that maps the knowledge from the Type I RDF graph to the corresponding CTI reports and ATT&CK techniques stored in the Type II knowledge graph, resulting in the Type III knowledge graph as illustrated in Fig. 3 (5) and (6). All graphs are stored in the knowledge base (7) which supports two options for back-end data storage and query processing:

- Neo4j graph database with normalized data queried using Cypher.
- GraphDB RDF store with normalized data queried using SPARQL.

3.3. Chatbot: Interface with human experts

We use CTI reports from NIST such as CVEs and CWEs to augment a pre-trained LLM to act as an interface with human cybersecurity experts, as shown in Fig. 2 (B). This part of the figure shows the functional blocks of the interface, which include the domain-specific database, the Q&A system, and the LLM. The next subsections describe three components within the LLM, the ReAct Agent (Section 3.3.1), another fine-tuned LLM as a sub-agent, and RAG (Section 3.3.2).

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

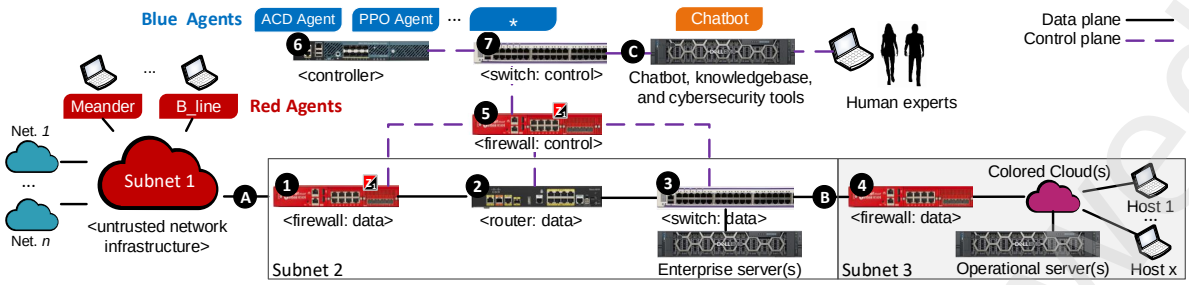


Figure 4: Network environment hosting three subnetworks, two red agents and one ACD agent (adapted from CAGE (2022)).

3.3.1. ReAct Agent

The ReAct agent uses prompts designed to guide its operation by integrating two tools: a RAG search system and a direct retrieval mechanism, both of which operate on the cybersecurity knowledge graphs implemented with a graph database management system (Neo4j). These knowledge graphs are implemented to reduce the number of LLM queries against the graphs that would be required with RAG tools, reducing token costs and query time. The system handles direct retrieval tool calls through predefined Neo4j Cypher queries. These include, for example, search queries for *"get_group"* (search for group name), *"get_software"* (search for software name), and *"search_for_technique_-by_id"* (search for techniques by input ID). Using this approach allows for quick cross-referencing of relationships by name or unique identifier based on the observations made by the agent. If the questions posed are ambiguous or lack information for a direct retrieval answer, the ReAct agent is prompted to *"opt"* for the more complex RAG enhanced search tools.

Specifically, the RAG tools include *"identify_mitigation"*, *"identify_cyber_attack_technique"*, and *"identify_-software"*. This segmentation allows the agent to improve search performance by selecting the most relevant search domain, allowing the embedding space to focus on a smaller, more targeted set of information. For each of these, the ReAct agent is asked to provide a question to be answered by the RAG tool. The question is then passed as input to the RAG pipeline. Consequently, each RAG tool can be thought of as a separate component accessible to the agent, with the ability to be individually tuned to meet the specific requirements of different node types, such as tactic, technique or mitigation.

3.3.2. Retrieval Augmented Generation (RAG)

For the retriever, we experiment with two embedding models. First, we used the Sentence Transformers framework, derived from Sentence-BERT (Reimers and Gurevych, 2019), to generate embeddings with a smaller output vector of 384 dimensions. In addition, we integrated the OpenAI embedding model, using the iteration *"text-embedding-ada-002"* as recommended by OpenAI, with a knowledge cut-off of September 2021. This model accommodates a maximum of 8,191 input tokens for a single sequence, producing an output vector of 1,536 dimensions. We used these two embedding models to process the names and descriptions of the node types, tactics, as well as attack and mitigation techniques as our text documents. These were then transformed into vectors and stored in the Neo4j vector store. Each vector is stored as a property of its corresponding node. The ReAct agent decides which vector store search index to use, i.e. which RAG pipeline, and comes up with different sub-questions as a planning mechanism leading to the overall goal of answering the user's question.

To improve the LLM's ability to provide accurate answers with augmented knowledge in the RAG pipeline, we prioritize nodes based on their relevance to the sub-question for effective information retrieval. This is achieved by computing the least cosine distance between all these embedding vectors and the question query vector. Therefore, our goal was to identify the "top_k=1" node that has the closest semantic meaning (or the node with the least cosine distance), ensuring that it is the most relevant and useful in response to our query coming from the ReAct agent. The closest node is then retrieved and automatically augmented with contextual information in natural language. For example, in the case of a particular mitigation strategy, all associated attack techniques are identified and aggregated based on the relationships established within the cybersecurity knowledge graph.

Before being returned to the ReAct agent, the answer is synthesized by querying an LLM with a specific prompt to generate an answer to the sub-question. The prompt also advises to reference the IDs of the source documents used. The full context retrieved is also saved as JavaScript Object Notation (JSON) files for later debugging. Finally, the answer to the sub-question asked by the ReAct agent is returned as an observation.

4. Experimental Setup: Network & Agents

This section describes the main components of the experimental setup used to train and evaluate our ACD agent in a gym environment, namely the network topology (Section 4.1), the two red agents (Section 4.2), our ACD agent (Section 4.3), and an example interaction with our chatbot (Section 4.3.2).

4.1. Network Topology

The network topology hosts 3 subnetworks inspired by the PCN paradigm used to connect a coalition of two or more nations, as shown in Fig. 4. This paradigm assumes

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

Table 2
High- and low-level action space for red agents.

High-level Action	Low-level Action	Description	Parameters	Output
Discover Remote Systems ATT&CK Tactic TA0007	Nmap Pingsweep ATT&CK Technique T1018	Scan for new hosts/IP adresses within a subnet	Subnet	IP addresses hosts that respond to ping
Discover Network Services ATT&CK Tactic TA0007	Portscan ATT&CK Technique T1046	Identifies responsive services on selected hosts	Subnet	Ports and service information
Exploit Network Services ATT&CK Tactic TA0008 ATT&ACK Technique T1210	Eternal Blue CVE-2017-0144	Windows SMB Remote Code Execution Vulnerability	IP Address, Port	Success/Failure Initial recon of host if successful
	BlueKeep CVE-2019-0708	Remote code execution vulnerability exists in Remote Services		
	HarakaRCE CVE-2016-1000282	Haraka version 2.8.8 and earlier can be vulnerable to command injection in the zip file plugin		
	FTP Directory Traversal CVE-2015-7601	Directory Traversal Vulnerability in FTP Server 2.0.7 or earlier		
	HTTPRFI	Remote File Inclusion using a crafted HTTP request		
	HTTPSRFI	Remote File Inclusion using a crafted HTTPS request		
	SSH Brute Force	Performs brute-force password attempts.		
	SQL Injection	Injected SQL Code Execution leaks system information		
Privilege Escalation ATT&CK Tactic TA0004	V4L2 Exlploit CVE-2019-18683	Windows NTLM Elevation of Privilege Vulnerability	Hostname	Success/Failure Internal information now available
	Juicy Potato CVE-2023-21746	Linux Privilege Vulnerability in drivers/media/platform/-vivid		
Impact	Stop Service ATT&CK Technique T1489	Disrupt network performance fulfilling red agent objective to deny operational services	Hostname	Success/Failure

that deployable tactical systems are exposed to adversaries through untrusted network infrastructure (subnet 1), such as commercial Internet and satellite links (A) in Fig. 4. This figure also shows a set of network devices forming the data plane for servers/hosts and a control plane hosting a set of autonomous agents used in the comparative evaluation reported later in section 5. This network topology has three firewalls (1), (4), and (5), a router (2), two switches (3) and (7), and a controller (6), which together form an exemplary critical communications infrastructure similar to a protected core network segment for a multinational military operation deployed in a particular region. For example, this network segment (subnetwork 2) may host at its edge a set of colored clouds with different security classifications (B) providing a set of operational services.

This investigation assumes that the control plane hosts SDN controllers with both connectivity and computational power to execute a set of autonomous agents implementing different AI models for a particular purpose (e.g. to detect and mitigate a particular type of cyberattack). All the network devices also implement SDN interfaces and the SDN switches are used to re-route Internet Protocol (IP) data flows to particular cybersecurity agents/functions leveraging virtualization and containerization (C). These interfaces are used by the agents to both monitor and actuate in the subnetworks and services by detecting cyberattacks and enforcing defense measures. The network topology, in Fig. 4, was simulated in a gym environment (CybORG (Standen et al., 2021)) to train and test our ACD agent. To do so, the network also hosted two red agents implementing the set of offensive actions listed in Table 2 and using attack strategies described in the next section.

4.2. Red Agents in the Gym Environment

Remember that the quality and performance of blue agents utilizing Deep Reinforcement Learning (DRL) models depend significantly on the realism and intelligence of the red agents deployed in the gym. The network topology in Fig. 4 also shows two red agents, *B_line* and *Meander*, deployed at user nodes connected to subnet 1 using untrusted network infrastructure such as commercial internet links (A). These links are used to access public services deployed within enterprise servers in subnet 2, which has a control plane hosting the ACD agents monitoring and protecting the network. Finally, subnet 3 hosts critical operational services and operational hosts which are protected in their security domain. The firewall (4) ensures that the operational services are accessible only through the operational hosts. As a result, the two red agents in subnet 1 cannot directly attack nodes and services in subnet 3.

The two red agents in the gym environment employ the set of offensive actions listed in Table 2 and their strategies are defined as follows:

- *Meander*: Aims to gain privileged access to all hosts in a subnet before moving on to the next one, eventually arriving at the operational server(s) in subnet 3. This strategy simulates an attacker that does not know the target network topology and operational services.
- *B_line*: Systematically explores each subnet directly targeting the operational servers using prior knowledge of the network topology and services. This strategy simulates an attacker that knows well the target network topology/services therefore minimizing the use of network mappers and port scanners.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

```
{`User2':
 {`CSKG Info':
  [{`Malicious': 0.9,
    `Tactic': TA0008,
    `Technique': T1210,
    `Malware': S0532,
    `CVE-ID': CVE-2017-0144,
    `Removed': False}],
 {`Files':
  [{`Malicious': 0.9,
    `File Name': 'cmd.exe',
    `Known File': <FileType.UNKNOWN: 1>,
    `Known Path': <Path.TEMP: 5>,
    `Path': 'C:/temp/',
    `Signed': False}],
 `Interface': [{
    `IP Address': IPv4Address('10.0.242.24')}],
 `Processes':
  [{`Connections':
   [{`local_address': IPv4Address('10.0.242.24'),
    `local_port': 139,
    `remote_address': IPv4Address('10.0.242.25'),
    `remote_port': 53143}]},
 `System info':
  {`Architecture': <Architecture.x64: 2>,
   `Hostname': 'User2',
   `OSDistribution': <OperatingSystemDistribution.
        WINDOWS_SVR_2008: 4>,
   `OSType': <OperatingSystemType.WINDOWS: 2>,
   `OSVersion': <OSVersion.W6_1_7601: 13>}},
   `success': <TrinaryEnum.TRUE: 1>}
```

Listing 2: Example of information about a cyberattack using the Cyber Security Knowledge Graph (CSKG).

For example, the training starts with the red agent initiating an attack sequence. This sequence begins with the compromise of a user host in subnet 1, followed by reconnaissance actions within subnet 2. These actions, specifically *'Discover Remote Systems'* and *'Discover Network Services'*, aim to identify active hosts and available services within the subnet. After successful enumeration of network services on a target host, the red agent attempts to exploit a vulnerability using one of the following techniques (ranked in ascending order of complexity): *'Eternal Blue'*, *'BlueKeep'*, *'HarakaRCE'*, *'FTP Directory Traversal'*, *'HTTPRFI'*, *'HTTPSRFI'*, *'SSH Brute Force'*, or *'SQL Injection'*. The exploit selection is probabilistic, with a 75% chance of employing the highest-ranked applicable exploit based on identified open ports.

Successful exploitation of a network service, followed by privilege escalation (achieved via *'V4L2 Exploit'* for Unix-based systems and *'Juicy Potato'* for Windows-based systems) within the control plane, grants the red agent access to the operational network (subnet 3) of the targeted cloud infrastructure. The red agent's goal is to disrupt a critical services residing on the operational server(s). This disruption is achieved through the successful execution of the *'StopService'* impact action.

4.3. Training the ACD Agent in the Gym

The ACD agent has been trained over 100,000 episodes and evaluated over 30-100 episodes, each episode being a set of discrete steps executed within a fixed time frame $[1, \cdots, 1000]$. Both red and ACD agents operate in a hierarchical action space composed of high-level and low-level actions. At each step, an agent selects a high-level action (column 1 in the 2 and 1 tables). This high-level action is then mapped to a specific, context-dependent low-level action (column 2 in tables 2 and 1), which defines the behavior of the agent. As a result, the agent can choose to *'Analyse'* node activities by executing low-level actions *'Monitor'*, *'Density Scout'*, *'SigCheck'*, and *'Query Cyber Security Knowledge Graph (CSKG)'* to access more precise information about a specific host distinguishing between malicious (red agent) and user activities.

Table 3
ACD agent rewards for successful red actions (CAGE, 2022).

Subnet	Host	Reward
Subnet 1	User Host	-0.1
Subnet 2	Enterprise Server	-1
Subnet 3	Operational Server	-1
Subnet 3	Operational Host	-0.1

Listing 2 shows example information collected by the agent as output from the *'Analyse'* action on the *'User2'* host in subnet 1. The output uses the Open Command and Control (OpenC2) language (OASIS, 2023) to gather information about the operating system, *'WINDOWS SERVER 2008'*, running processes, '10.0.242.24' on local port '139', file information, *'C:/temp/cmd. exe'* with potential malware (density 0.9) received from low-level actions using *'DensityScout'* and/or *'SigCheck'*, and information about a potential cyber threat chain: Tactic 'TA008', Technique 'T1210', Exploit *'CVE-2019-0144'*, Installed Malware 'S0532'. Knowledge extraction is performed using a Deep Neural Network (DNN) classifier on the knowledge graph resulting from the knowledge base. The classifier implements a LSTM that wraps the monitored log data (*'Interfaces'*, *'System Info'*, and *'Processes'*) into a system state represented by a dense vector, which is then embedded into the knowledge graph using Node2Vec and the cosine similarity distance measure.

As a result, the ACD agent uses the output of the *'Analyse'* action to react to ongoing malicious activities. The agent implements the capabilities to remove red access (effective only before *'Privilege Escalation'* actions by the red agent) and *'Restore systems'* actions to a backup configuration. However, the actions to restore the system action disrupt user activities. Moreover, the ACD agent can choose from a set of *'Decoy'* actions, namely *'DecoyApache'*, *'DecoyFemitter'*, *'DecoyHarakaSMPT'*, *'DecoySmss'*, *'DecoySSHD'*, *'DecoySvchost'*, and *'DecoyTomcat'* defined in Table 1. These decoy services are honey pots used to delay the red agent because any attempt to exploit these decoys result in automatic failure. The observations of the ACD agent contain system information relevant to the previous action according to OpenC2 Language collected either by automated execution of the monitoring actions or selection of the *'Analyse'* actions. Both outputs are wrapped to a single dense observation vector to be used by the DRL model.

The optimal strategy for the ACD agent is computed by an actor-critic DRL model using Proximal Policy Optimization (PPO), within the architecture in Fig. 2 defined earlier in Section 3. This model is used to select the best set of actions, also called policy, within the agent's action

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

Table 4
Rewards for red and blue agents (CAGE, 2022).

Agent	Host	Action	Reward
Red	Operational Server	Impact	-10
ACD	Any	Restore	-1

space (Table 1), to defend the network environment. The actor-critic neural network consists of 2 fully connected hidden layers using Rectified Linear Unit (ReLU) activation functions for the hidden layers and softmax activation for the final actor layer. We enhanced the agent to be allowed to have one single decoy action per host, which is selected greedily from available decoys on that host. Moreover, we padded the observation vector with information about the scanning state, indicating if the current host was already scanned in a previous action.

The reward system penalizes the ACD agent for any administrative access gained by the red agent in both enterprise and operational servers, as indicated by negative rewards in Table 3. Complementing, Table 4 shows that the rewards are scaled according to the host's system importance and actions performed on the operational server. As a result, the final score for an ACD agent is the cumulative reward across the scenario during training or testing in the gym environment.

4.3.1. Exemplary Attack Scenario in the Gym

An example attack scenario is shown in figure. 5. First, the red agent performs *'Remote System Discovery'* (1) and finds the host 'User2'. After running *'Network Service Discovery'* (2), it finds a service with IP '10.0.242.24' that is vulnerable to the 'Eternal Blue' exploit (3). So the red agent decides to exploit the service with *Lucifer* and installs malicious code to gain administrative (root) access to the operating system. In parallel, the ACD agent was installing decoy services on other hosts and performing monitoring and analysis actions on the target host/user (*'User2'*) while the red agent was performing actions (1-3), so the agent is able to *'Remove'* (7) the malicious code. Otherwise, the red agent can proceed with *'Privilege Escalation'* (4) and possibly gain administrative access to the *'User2'* host. In this case, the only option for the ACD agent is to *'Restore'* the machine (6) to a backup state. Otherwise, i.e. by selecting another action, the red agent can perform a *'Impact* action on the service with IP '10.0.242.24:139' and finally stop the service.

4.3.2. Exemplary Interaction with the Chatbot

Our chatbot can explain the set of actions performed in attack scenarios in the gym environment. The system uses a pre-trained LLM augmented with cybersecurity knowledge graphs to explain the attack chain and to provide insights into specific characteristics in a sequence of offensive and defensive actions. For example, Fig. 6 illustrates an interaction between a human cybersecurity expert and our chatbot. The human expert writes a prompt asking the system to clarify the relationship between two high-level actions, *Exploitation of Remote Services* and *Remote System Discovery*,

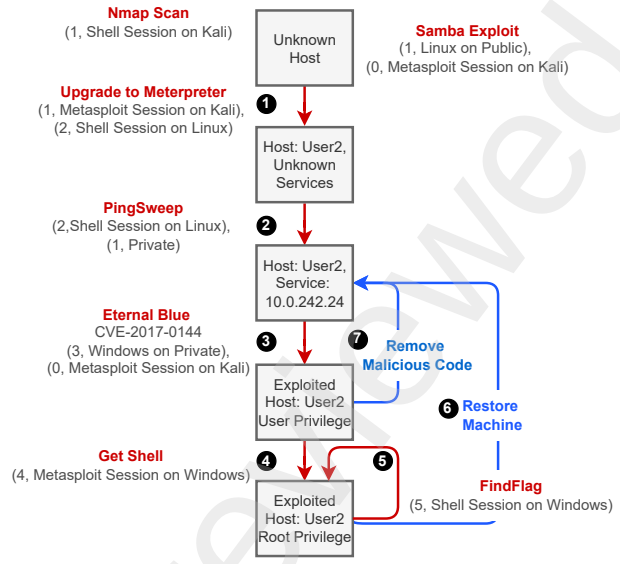


Figure 5: Example of interaction between red and blue agents in the gym.

emphasizing how these techniques complement each other in cyberattack scenarios. The top shows the user prompt followed by the ReAct prompt asking the LLM to act as a cybersecurity expert. The tool descriptions are presented as natural language text, detailing the functionalities of the tools discussed earlier in Section 3.3. The format instructions guide the LLM to maintain a consistent structure of Thought (T), Action (A), and Action Input (AI) pairs. After generating the output, the observation (O) is appended to it, as illustrated in point of time (1). The chatbot keeps track of its past interactions by saving the outputs in the prompt (2). This process is repeated two more times until the final answer is produced (3), providing the human expert with a clearer explanation of the attack techniques.

5. Experimental Results

This section evaluates the BERT-based model used to build the Type II CTI knowledge graph and the performance of our ACD agent using the Type III knowledge graph against a red agent with two strategies (Section 5.2), as well as the generation and retrieval of our chatbot using a pre-trained LLM augmented with cybersecurity knowledge graphs (Section 5.3).

5.1. Mapping CVEs to MITRE ATT&CK

The performance of five LLMs mapping CTI reports to the MITRE ATT&CK database are quantified and compared using a set of metrics (precision, recall, F1 micro, and F1 Macro). To do so, we trained three encoders for BERT-based models fine-tuned with cybersecurity datasets, as well as GPT-4 and GPT-4o as a baseline for the comparison. First, A Pretrained Language Model for Cyber Security Text (SciBERT) is a BERT-based model pre-trained on a cybersecurity corpus. This model was fine-tuned using the Masked Language Modeling (MLM) process, where the model learns to guess randomly masked words from

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

Main prompt	USER Prompt: Could you clarify how the 'Exploitation of Remote Services' (T1210) and 'Remote System Discovery' (T1018) work together in cyber attacks, and what methods are typically used for this process? SYSTEM: (ReAct prompt) From now on, you are a cybersecurity expert who gives detailed answers to questions as best as you can. You have access to the following tools: {tool descriptions} {format instructions adapted from (Yao et al., 2023)} Begin! {format instructions} {user question}→[T][A][AI]
1	SYSTEM: (Thought [T], Observation [O] pruned) [A] Action: identify_cyber_attack_technique [AI] Action Input: Exploitation of Remote Services →[O]
2	SYSTEM: (Thought [T], Observation [O] pruned) [A] Action: identify_cyber_attack_technique [AI] Action Input: Remote System Discovery →[O]
3	SYSTEM: Output to USER The 'Exploitation of Remote Services' (T1210) and 'Remote System Discovery' (T1018) techniques work together in cyber attacks. Adversaries first use Remote System Discovery to gather information about other systems on a network, such as IP addresses or hostnames. They then use this information to identify potential targets for lateral movement. Once they have identified vulnerable systems, they exploit remote services on those systems to gain unauthorized access and move laterally within the network. Common services such as SMB, RDP, MySQL, and web server services are often targeted for exploitation.

Figure 6: User question, ReAct prompt, and three Chatbot responses.

a variety of security texts such as Cisco security reports, the NIST website, and books on cybersecurity and hacking techniques. Second, SecRoBERTa was fine-tuned on the same cybersecurity corpus as SciBERT, but, as its name suggests, the base model is RoBERTa, which is an optimized variation of BERT. This variant was trained on a larger corpus compared to BERT, had the benefit of better hyper-parameters found to be suboptimal in the original BERT, and had a different, dynamic word masking strategy during training.

Third, Cyber Security BERT (CyBERT) is a domain-specific BERT model that has been fine-tuned with a large corpus of textual unlabeled cybersecurity data. It was fine-tuned starting from the BERT base model with MLM, which is similar to SciBERT. More specifically, the first step consisted of extending the model's vocabulary with specific cybersecurity words. Once the vocabulary was extended and the embedding space was updated, the model was trained to guess randomly masked words from the specific cybersecurity corpus. The main difference between SciBERT and CyBERT lies in the data the two models were fine-tuned on. More specifically, CyBERT was trained using a corpus of labeled sequences from Industrial Control Systems (ICSs)' device documentation collected across a wide range of vendors and devices, while SciBERT was trained using a large number of online cybersecurity-related text data, including books, blogs, news, security reports, videos (subtitles), journals and conferences, white papers, tutorials, and survey papers.

Table 5
Performance metrics from five LLMs mapping CVEs to MITRE ATT&CK techniques.

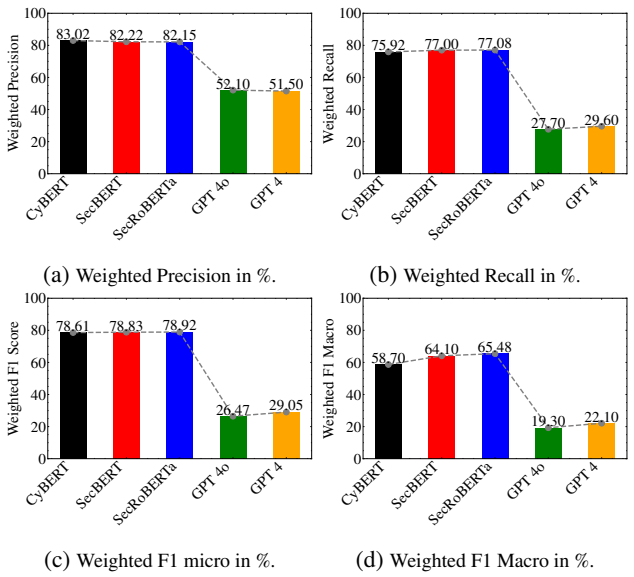
Model	Precision (SD)	Recall (SD)	F1 Micro (SD)	F1 Macro (SD)
CyBERT	**83.02** (1.10)	75.92 (1.35)	78.61 (1.20)	58.70 (4.49)
SecBERT	82.22 (0.44)	77.00 (0.66)	78.83 (0.58)	64.10 (2.40)
SecRoBERTa	82.15 (0.39)	**77.08** (1.25)	**78.92** (0.72)	**65.48** (2.00)
GPT 4o	52.10 (0.01)	27.70 (0.01)	26.47 (0.01)	19.30 (0.01)
GPT 4	51.50 (0.01)	29.60 (0.01)	29.05 (0.01)	22.10 (0.01)

These five models were used to encode the CVE text description contained in the single (Micro) and multi-label annotation datasets (Macro) generated using the architecture from Fig. 3.2.2. The input layer is followed by a pooling and a linear layer with 50 output nodes, one for each class within the 50 most common techniques of 2024 proposed by MITRE, coupled with a loss function (BCEWithLogitsLoss). The output represents a probability for each class, based on which it is decided if the CVE can be exploited using a given offensive technique. For the multi-label task, we computed confusion matrices for each class. In training, the number of epochs was chosen by observation of the behavior in longer experiments based on when the validation loss converged to a constant value. A learning rate search based on the best average validation score for each model was performed for all models to find the optimal value. For all transformers, we used the AdamW optimizer, the batch size was set to 16, and the learning rate during training was set to 3×10^{-5}. No input truncation was needed, since the longest weakness was shorter than the 512 token input limit set by BERT.

Table 5 highlights the per-technique results for five different LLMs including the best performing model, SecRoBERTa. The reported results have been computed after the model has been trained on the entire training and validation dataset and tested on the fixed test dataset. The models with the best F1-micro scores were SecRoBERTa (78.92%), SciBERT (78.83%), and CyBERTs (78.61%). Moreover, the analysis of the multi-label macro dataset showed high precision for CyBERT (83.02%), high recall for SecRoBERTa (77.08%), and a high F1-macro for SecRoBERTa (65.48%), as also illustrated in the bar-plots in Fig.7 (a, b, c and d). These results suggest that the models effectively extracted relevant information during training and demonstrated some ability to generalize to unseen data, indicating the significant potential for extracting valuable insights from the textual descriptions of CVEs. However, despite fine-tuning, the models still showed some limitations in addressing the substantial class imbalance inherent in the dataset.

Given the relatively large number of classes for the current multilabel classification task (macro), the macro F1 score of 78.92%, which gives equal value to each class, shows that even if the model naturally has the tendency to better classify the class with the largest number of samples, it still differentiates between the rest of the classes, thus extracting relevant linguistic features from the CVE's description. As shown in Table 5, GPT-4 and GPT-4o show

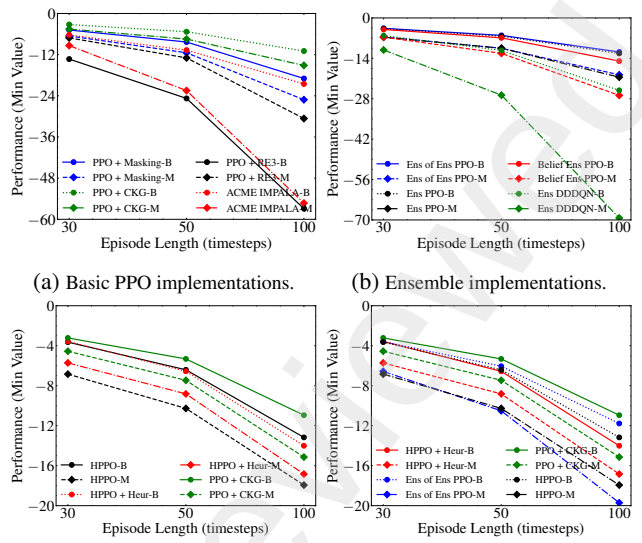
This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836



(a) Weighted Precision in %. (b) Weighted Recall in %.

(c) Weighted F1 micro in %. (d) Weighted F1 Macro in %.

Figure 7: Precision (a), recall (b), F1 micro (c), and F1 Macro (d) from five LLMs mapping CVEs to offensive techniques.



(a) Basic PPO implementations. (b) Ensemble implementations.

(c) Hierarchical PPO. (d) Best performing implementations.

Figure 8: Rewards for ACD Agents using different DRL Algorithms against two red agent strategies (B) and (M).

low performance in a zero-shot setup compared to the fine-tuned models. Adding a short description of each technique in the prompt did not improve the results, thereby leading to slightly worse performance.

Finally, we evaluated the LSTM classifier for mapping the system information present in the Type I knowledge graph to the Type II CTI graph, as shown earlier in Fig. 3 (6). The classifier was evaluated with three different sets of features that were randomly selected to construct a node embedding of the log data. Four quantitative metrics (accuracy, precision, recall, and F1 score) were computed during both training and testing of the classifier using the cybersecurity knowledge graph mapping 100,000 log samples (80% for training and 20% for testing). The log samples were monitored from the network topology simulated in the gym environment and mapped to MITRE ATT&CK tactics and techniques. High accuracy of 98.30% ($\pm$ 0.07) and high F1 scores of 97.78% ($\pm$ 0.06) indicate that even with 6 random features from the log data, the classifier can learn to map system state to a node in the cybersecurity knowledge graph.

5.2. Evaluation of the ACD Agent

Table 6 lists the cumulative rewards of 12 DRL algorithms, including our solution using PPO and cybersecurity knowledge graphs (PPO + CSKG). This table shows the rewards observed in episodes of three lengths (30, 50, and 100 timesteps) and against two red agents: *B_line* (B) and *Meander* (M). These results indicate that our solution outperforms the other algorithms over the three episode lengths. Complementing, Fig. 8 shows four plots comparing the cumulative rewards as a function of number of steps within episodes for *basic PPO implementations* (a), *ensemble implementations* (b), *hierarchical PPO* (c) and the *best performing algorithms* (d).

Remember that the basic PPO implementations lack specific enhancements like a single decoy plus buffer per host and scanning state integration. All DRL implementations were evaluated against two red agent strategies following a training program of 100,000 episodes, with each episode comprising 1,000 steps. After training, the performance evaluation was carried out over 1,000 episodes, each consisting of 100 steps, with cumulative rewards computed at 20-step intervals. Our experimental results indicate that the ACD agent converged to an average cumulative reward of approximately -13 for the *B_line* strategy and -16 for the *Meander* strategy. When compared to the standard PPO implementations, these results demonstrate that the specialized modifications to the ACD agents (namely the single decoy strategy, the decoy buffer, and state scanning) lead to a significant improvement in rewards.

The ACD agents employed a consistent initial action pattern in response to both the *B_line* and *Meander* attack strategies. In both cases, the agents prioritized deploying two specific decoy actions: *'Fermitter User2'* and *'Tomcat User2'*. However, the choice of the third decoy action was contingent upon the red agent's second action. A key distinction emerged by the fourth action, where the *Meander* strategy was characterized by two consecutive scans on different hosts, differentiating it from the *B_line* strategy. Based on this observation, we adopted a strategy of training two distinct agents, pre-configuring their first three actions as *'Fermitter User2'*, *'Tomcat User2'*, and *'SMTP Ent0'*.

5.3. Evaluation of the augmented LLM

This section discusses the evaluation of our chatbot by describing the features from the knowledge-graph (Section 5.3.1) and quantitative results for generation (Section 5.3.2) and retrieval (Section 5.3.3).

5.3.1. Evaluation Setup

The knowledgebase used to augment the LLM is created from a CTI-Classification Initiative (Orbinato et al., 2022)

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

Table 6
Performance of DRL agents under different episode lengths (30, 50 and 100 timesteps) and two red agent strategies (B_Line and Meander).

Blue Agent	Episode Length & Red Agents					
	30 timesteps		50 timesteps		100 timesteps	
	B-Line	Meander	B-Line	Meander	B-Line	Meander
PPO + CSKG	**[-3.22, -4.18]**	**[-4.55, -4.35]**	**[-5.32, -5.51]**	**[-7.46, -7.99]**	**[-10.95, -10.54]**	**[-15.14, -15.30]**
HPPO + Heur	[-3.57, -3.36]	[-5.72, -5.56]	[-6.56, -6.25]	[-8.82, -8.56]	[-14.01, -13.51]	[-16.84, -16.35]
Ens of Ens PPO	[-3.59, -3.36]	[-6.57, -6.37]	[-6.05, -5.65]	[-10.51, -10.16]	[-11.78, -11.01]	[-19.71, -19.05]
HPPO	[-3.64, -3.48]	[-6.84, -6.67]	[-6.41, -6.00]	[-10.27, -9.99]	[-13.17, -12.39]	[-17.94, -17.36]
Ens PPO	[-3.72, -3.50]	[-6.67, -6.46]	[-6.30, -5.91]	[-10.50, -10.15]	[-12.28, -11.53]	[-20.53, -19.77]
Belief Ens PPO	[-3.99, -3.75]	[-6.66, -6.42]	[-6.90, -6.57]	[-12.28, -11.74]	[-14.95, -14.30]	[-26.83, -23.35]
PPO TL	[-5.71, -5.36]	[-6.51, -6.29]	[-10.03, -9.45]	[-10.37, -10.04]	[-20.00, -19.14]	[-19.05, -18.26]
PPO + Masking	[-4.82, -4.38]	[-6.55, -6.35]	[-8.32, -7.51]	[-11.46, -10.99]	[-18.95, -16.54]	[-25.14, -23.30]
PPO (baseline)	[-4.93, -4.58]	[-8.17, -7.90]	[-8.42, -7.83]	[-13.19, -12.63]	[-17.84, -16.55]	[-23.45, -22.44]
ACME IMPALA	[-6.25, -5.74]	[-9.36, -9.17]	[-10.69, -9.81]	[-22.48, -22.06]	[-20.53, -18.90]	[-55.28, -54.29]
PPO + RE3	[-13.29, -11.68]	[-7.11, -6.87]	[-24.76, -21.89]	[-12.94, -11.58]	[-56.88, -51.14]	[-30.62, -26.75]
Ens DDDQN	[-6.12, -5.61]	[-11.10, -10.70]	[-11.26, -10.37]	[-26.76, -25.82]	[-25.08, -22.84]	[-69.31, -66.47]

dataset, which consists of adversarial techniques and unstructured sentences from the MITRE ATT&CK framework. As a result, the chatbot can use this knowledge to explain events in the network (generation) and retrieve information to support experts/agents defending the network. Our approach involves randomly picking one entry from each main technique row listed in the dataset, while excluding sub-techniques to avoid added complexity. Before starting the evaluation phase, we had to remove 17 techniques that were missing from our graph database, ensuring our analysis focused only on retrievable techniques. As a result, our knowledge-graph included 178 attack techniques, 12 tactics, 43 mitigation strategies, 138 threat groups, and 717 software nodes.

This dataset is further enriched with meaningful queries generated with a state-of-the-art LLM (ChatGPT's GPT-4, knowledge cutoff April 2023). Each question is designed to be answerable by a specific sentence extracted from the CTI report included in the dataset. Additionally, we validate the appropriateness of these questions by examining each one to ensure it logically resulted in the provided answer, thereby maintaining the quality and relevance of our dataset for this investigation. Subsequently, we select the first five rows for our evaluation. This is the primary limitation of the experiment. Then we selected Ragas as our evaluation framework for RAG. Ragas can do automatic evaluation (with some exceptions like *Semantic Similarity*, where we choose the sentence of CTI in our test dataset) for RAG evaluation, advanced by the LLMs' own ability to help evaluate (Chang et al., 2024). Ragas is introduced in Section 2.2.3 and we choose version 0.0.22 (Es et al., 2024; Ragas, 2024). Although these automatic metrics provide valuable insights, they should be interpreted carefully due to potential biases and hallucinations in LLM assessments (Rawte et al., 2023; Wang et al., 2023). In the implementation and experimentation of this framework, all prompts are evaluated using the *gpt-3.5-turbo-16k* model, which is available through the OpenAI Application Programming Interface (API) (OpenAI, 2024).

5.3.2. Generation Results

Fig. 9 shows the scores for *Answer Relevancy*, *Faithfulness*, and *Semantic Similarity*, comparing the performance of ReAct+RAG chatbot to that of Zero-Shot implementation. In this comparison, Zero-Shot acts as the baseline, representing the use of an LLM without any additional enhancements. The average scores for *Answer Relevancy* were 0.96 for ReAct+RAG and 0.99 for Zero-Shot. For *Faithfulness*, ReAct+RAG scored 0.80, while Zero-Shot scored 0.23. Regarding *Semantic Similarity*, ReAct+RAG achieved a score of 0.91, compared to 0.89 for Zero-Shot.

While the Zero-Shot approach yields results similar to ReAct+RAG in terms of *Semantic Similarity*, the ROUGE-1 analysis reveals that ReAct outperforms Zero-Shot in structural similarity, with ReAct scoring 0.53 compared to Zero-Shot's 0.25. This example underscores the importance of incorporating both semantic and structural similarity metrics for a well-rounded evaluation of LLMs. Furthermore, this is evidenced by the *Faithfulness* score, as the Zero-Shot method often produces text that, while semantically similar, falls outside the context and can be factually incorrect, making it difficult to substantiate. As a result, it can be concluded that the ReAct+RAG agent is a more reliable choice for producing contextually validated outputs, though further investigation could provide additional insights.

5.3.3. Retrieval Results

For retrieval, especially within the embedding space, we used sentence transformer embeddings as described above. While this introduces some variation into the experiment, we observe consistent effects across models. We observed a *Context Precision* score of 0.79, which could potentially be improved by refining tools and fine-tuning prompts to improve focus and minimize retrieval of irrelevant context in

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

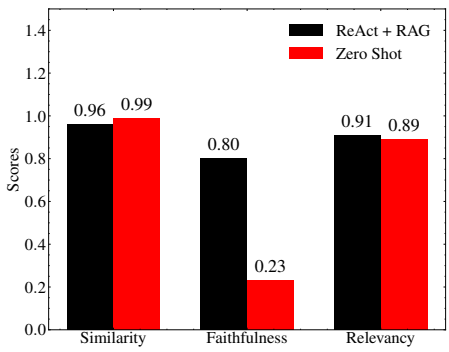


Figure 9: Generation Results: ReAct+RAG against Zero-Shot.

response to action input. For the *Context Recall* metric, we observed a score of 1.0, demonstrating the model's ability to retrieve relevant instances on demand. The F1 score is around 0.88, indicating that our chatbot has learned the context and content from the data in the knowledge base. A key strength of the retrieval system is its direct support for semi-transparent explicability. By linking the LLM output to external context, the system allows users to understand the reasoning behind its answers. This transparency builds trust and usability by revealing the origin of the retrieved documents or data.

6. Conclusion

This paper introduced and evaluated an architectural design for an Autonomous Cyber Defense Agent trained to enforce a set of defensive actions, along with a chatbot that uses a pre-trained LLM to function as interface with human cybersecurity experts. Our hybrid design combines a DRL model with a Type III knowledge graph that represents information about the existing network infrastructure and possible attacker tactics, techniques, and system vulnerabilities present in the network. An augmented LLM with CTI reports acts as an interface for human operators to retrieve known information and generate answers to questions about security risks. In addition, we proposed a BERT-based encoder model to map CVEs to MITRE ATT&CK techniques to build a Type II knowledge graph representing cyberthreat intelligence covering open source data from MITRE ATT&CK, NIST, and MITRE D3FEND. This graph is combined with the Type I RDF network graph to form the Type III graph, which is used for risk analysis of the system infrastructure and by the DRL agent to find its optimal policy.

We implemented and tested a prototype of our ACD agent design in a gym environment designed to train and test DRL models to automate defenses in a network with two security domains hosting enterprise and operational services. The network environment included red agents capable of detecting and exploiting vulnerabilities in operating systems and services using two different strategies. The information monitored by the ACD agent is classified using the cybersecurity knowledge graph to trigger an action to detect vulnerabilities in services in the network. The experimental results reported in this paper indicate that DRL algorithms using threat intelligence are a promising technique for automating decision making within ACD agents. In addition, the retrieval and generation performance of our chatbot was compared to zero-shot prompting using a base LLM without augmentation. Our prototype (ReAct+RAG) produced reliable responses to queries using the CTI reports within the knowledge base, demonstrating its potential for practical use.

As future work, we plan to extend the ACD agent architecture to include a team of specialized agents that collaborate to defend the network against a larger set of cyber attacks. The goal is to use MARL to allow distributed agents to collaborate as a team, using the control plane to communicate events observed in different network segments. The performance and quality of the blue team depends on how realistic the red team is during training and testing. Therefore, we also plan to design, implement, and test a hierarchy of specialized red agents capable of performing realistic attacks during training. The ultimate goal is to design, evaluate, and demonstrate a methodology for training/testing robust and secure ACD agents that automate a set of tasks to defend critical network infrastructure.

CRediT authorship contribution statement

Johannes Loevenich: Writing – original draft, Writing – review & editing, Conceptualization, Data curation, Methodology, Software. **Erik Adler:** Writing - original draft, Writing – review & editing, Data curation, Conceptualization, Methodology, Software. **Tobias Hürten:** Writing – review & editing, Conceptualization. **Roberto Rigolin F. Lopes:** Writing - original draft, Writing – review & editing, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used ChatGPT (GPT-4o) in order to improve text (grammar and sentence structure). After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Adlakha, V., BehnamGhader, P., Lu, X.H., Meade, N., Reddy, S., 2023. Evaluating Correctness and Faithfulness of Instruction-Following Models for Question Answering, in: arXiv, pp. 1–18. doi:10.48550/arXiv.2307.16877.

Ampel, B., Samtani, S., Ullman, S., Chen, H., 2021. Linking Common Vulnerabilities and Exposures to the MITRE ATT&CK Framework: A

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

Self-Distillation Approach, in: arXiv, pp. 1–5. doi:10.48550/arXiv.2108.01696.

Banerjee, D., Singh, P., Avadhanam, A., Srivastava, S., 2023. Benchmarking LLM powered Chatbots: Methods and Metrics. arXiv:2308.04624.

Becker, N., Reti, D., Ntagiou, E.V., Wallum, M., Schotten, H.D., 2024. Evaluation of Reinforcement Learning for Autonomous Penetration Testing using A3C, Q-learning and DQN, in: arXiv, pp. 1–12. doi:10.48550/arXiv.2407.15656.

Booker, L.B., Musman, S.A., 2020. A model-based, decision-theoretic perspective on automated cyber response. arXiv:2002.08957.

CAGE, 2022. TTCP CAGE Challenge 2, in: AAAI-22 Workshop on Artificial Intelligence for Cyber Security (AICS).

Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P.S., Yang, Q., Xie, X., 2024. A Survey on Evaluation of Large Language Models. ACM Trans. Intell. Syst. Technol. 15. doi:10.1145/3641289.

Es, S., James, J., Espinosa Anke, L., Schockaert, S., 2024. RAGAs: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, St. Julians, Malta. pp. 150–158. doi:10.48550/arXiv.2309.15217.

Fard, N.E., Selmic, R.R., Khorasani, K., 2023. A review of techniques and policies on cybersecurity using artificial intelligence and reinforcement learning algorithms. IEEE Technology and Society Magazine 42, 57–68. doi:10.1109/MTS.2023.3306540.

Gao, C., Wang, Y., 2021. Reinforcement learning based self-adaptive moving target defense against DDoS attacks. Journal of Physics: Conference Series 1812, 12–39. doi:10.1088/1742-6596/1812/1/012039.

Goertzel, B., 2014. Artificial General Intelligence: Concept, State of the Art, and Future Prospects. Journal of Artificial General Intelligence 5, 1–48. doi:10.2478/jagi-2014-0001.

Grigorescu, O., Nica, A., Dascalu, M., Rughinis, R., 2022. CVE2ATT&CK: BERT-Based Mapping of CVEs to MITRE ATT&CK Techniques. Algorithms 15. doi:10.3390/a15090314.

Haddad, A., Aaraj, N., Nakov, P., Mare, S.F., 2023. Automated Mapping of CVE Vulnerability Records to MITRE CWE Weaknesses, in: arXiv, pp. 1–15. doi:10.48550/arXiv.2304.11130.

Hamid, T.K., Maple, C., Sant, P., 2012. Methodologies to develop quantitative risk evaluation metrics. International Journal of Computer Applications 48, 17–24. doi:10.5120/7416-0413.

Huang, L., Zhu, Q., 2019. Adaptive Strategic Cyber Defense for Advanced Persistent Threats in Critical Infrastructure Networks. SIGMETRICS Perform. Eval. Rev. 46, 52–56. doi:10.1145/3305218.3305239.

Kiely, M., Bowman, D., Standen, M., Moir, C., 2023. On autonomous agents in a cyber defence environment, in: arXiv, pp. 1–8. doi:10.48550/arXiv.2309.07388.

Kott, A., Theron, P., Drasar, M., Dushku, E., LeBlanc, B., Losiewicz, P., Guarino, A., Mancini, L.V., Panico, A., Pihelgas, M., et al., 2019. Autonomous Intelligent Cyber-Defense Agent (AICA) Reference Architecture, Release 2.0. Technical Report. CCDC Army Research Laboratory.

Kuppa, A., Aouad, L., Le-Khac, N.A., 2021. Linking CVE's to MITRE ATT&CK Techniques, in: Proceedings of the 16th International Conference on Availability, Reliability and Security, Association for Computing Machinery, New York, NY, USA. pp. 1–12. doi:10.1145/3465481.3465758.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, in: Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 9459–9474. doi:10.5555/3495724.3496517.

Lin, C.Y., 2004. ROUGE: A Package for Automatic Evaluation of summaries, in: Proceedings of the ACL Workshop: Text Summarization Braches Out, p. 10.

Liu, Y., Guo, Y., Ranjan, R., Chen, D., 2024. Optimization of mitigation deployment using deep reinforcement learning over an enhanced att&ck. Computing 106, 4015–4038. doi:10.1007/s00607-024-01344-4.

Loevenich, J.F., Adler, E., Bécue, A., Velazquez, A., Wrona, K., Boshnakov, V., Falkcrona, J., Nordbotten, N., Worthington, O.L., Röning, J., Lopes, R.R.F., 2024a. Training Autonomous Cyber Defense Agents: Challenges & Opportunities in Military Networks, in: MILCOM 2024 - 2024 IEEE Military Communications Conference (MILCOM), Washington, USA. pp. 158–163. doi:10.1109/MILCOM61039.2024.10773923.

Loevenich, J.F., Adler, E., Mercier, R., Velazquez, A., Lopes, R.R.F., 2024b. Design of an Autonomous Cyber Defence Agent using Hybrid AI models, in: 2024 International Conference on Military Communication and Information Systems (ICMCIS), Koblenz, Germany. pp. 1–10. doi:10.1109/ICMCIS61231.2024.10540988.

Loevenich, J.F., Hürten, T., Spelter, F., Adler, E., Braun, J., Moxon, L., Gourlet, Y., Lefeuvre, T., Lopes, R.R.F., 2024c. Towards Robust and Secure Autonomous Cyber Defense Agents in Coalition Networks, in: MILCOM 2024 - 2024 IEEE Military Communications Conference (MILCOM), Washington, USA. pp. 152–157. doi:10.1109/MILCOM61039.2024.10773821.

Lopes, R.R.F., Loevenich, J.F., Wrona, K., Rettore, P.H.L., Falkcrona, J., Mathews, J., Nordbotten, N., Vasilache, B., Lampe, T., Worthington, O.L., Röning, J., 2023. Accelerating NATO transformation with SnTEE: Experiments with network security function virtualization in coalition networks, in: 2023 International Conference on Military Communications and Information Systems (ICMCIS), pp. 1–9. doi:10.1109/ICMCIS59922.2023.10253524.

Mavi, V., Jangra, A., Jatowt, A., 2022. Multi-hop Question Answering and Generation, in: arXiv, pp. 1–136. doi:10.48550/arXiv.2204.09140.

MITRE, 2024a. ATT&CK Knowledge Base. https://attack.mitre.org. [Accessed 25-11-2024].

MITRE, 2024b. MITRE Annotation Toolkit (MAT). http://mat-annotation.sourceforge.net/. [Accessed 01-12-2024].

OASIS, 2023. OpenC2 Language Specification. https:https://github.com/oasis-tcs/openc2-oc2ls. [Accessed 20-11-2024].

OpenAI, 2024. ChatGPT API. https://platform.openai.com/docs/overview. [Accessed 04-01-2024].

Orbinato, V., Barbaraci, M., Natella, R., Cotroneo, D., 2022. Automatic mapping of unstructured cyber threat intelligence: An experimental study: (practical experience report), in: 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE), pp. 181–192. doi:10.1109/ISSRE55969.2022.00027.

Ovadia, O., Brief, M., Mishaeli, M., Elisha, O., 2023. Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs. arXiv:2312.05934.

Papineni, K., Roukos, S., Ward, T., Zhu, W.J., 2002. BLEU: a method for automatic evaluation of machine translation, in: Proceedings of the 40th Annual Meeting on Association for Computational Linguistics, Association for Computational Linguistics, USA. p. 311–318. doi:10.3115/1073083.1073135.

Peng, S., Hu, X., Zhang, R., Tang, K., Guo, J., Yi, Q., Chen, R., Zhang, X., Du, Z., Li, L., Guo, Q., Chen, Y., 2024. Causality-driven hierarchical structure discovery for reinforcement learning, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA. doi:10.5555/3600270.3601729.

Piplai, A., Kotal, A., Mohseni, S., Gaur, M., Mittal, S., Joshi, A., 2023. Knowledge-enhanced neurosymbolic artificial intelligence for cybersecurity and privacy. IEEE Internet Computing 27, 43–48. doi:10.1109/MIC.2023.3299435.

Ragas, 2024. Evaluation framework for your Retrieval Augmented Generation (RAG) pipelines. https://github.com/explodinggradients/ragas. [Accessed 06-01-2024].

Rawte, V., Sheth, A., Das, A., 2023. A Survey of Hallucination in Large Foundation Models, in: arXiv, pp. 1–11. doi:10.48550/arXiv.2309.05922.

Reimers, N., Gurevych, I., 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, in: arXiv, pp. 1–11. doi:10.48550/arXiv.1908.10084.

Rezende, D.J., Danihelka, I., Papamakarios, G., Ke, N.R., Jiang, R., Weber, T., Gregor, K., Merzic, H., Viola, F., Wang, J., Mitrovic, J., Besse, F., Antonoglou, I., Buesing, L., 2020. Causally Correct Partial Models for Reinforcement Learning, in: arXiv, pp. 1–28. doi:10.48550/arXiv.2002.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836

02836.

Schulhoff, S., Ilie, M., Balepur, N., Kahadze, K., Liu, A., Si, C., Li, Y., Gupta, A., Han, H., Schulhoff, S., Dulepet, P.S., Vidyadhara, S., Ki, D., Agrawal, S., Pham, C., Kroiz, G., Li, F., Tao, H., Srivastava, A., Costa, H.D., Gupta, S., Rogers, M.L., Goncearenco, I., Sarli, G., Galynker, I., Peskoff, D., Carpuat, M., White, J., Anadkat, S., Hoyle, A., Resnik, P., 2024. The Prompt Report: A Systematic Survey of Prompting Techniques, in: arXiv, pp. 1–76. doi:10.48550/arXiv.2406.06608.

Standen, M., Lucas, M., Bowman, D., Richer, T.J., Kim, J., Marriott, D., 2021. CybORG: A Gym for the Development of Autonomous Cyber Agents, in: arXiv, pp. 1–7. doi:10.48550/arXiv.2108.09118.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA. p. 6000–6010. doi:10.5555/3295222.3295349.

Velazquez, A., Lopes, R.R.F., Bécue, A., Loevenich, J.F., Rettore, P.H.L., Wrona, K., 2023a. Autonomous cyber defense agents for NATO: Threat analysis, design, and experimentation, in: MILCOM 2023 - 2023 IEEE Military Communications Conference (MILCOM), pp. 207–212. doi:10.1109/MILCOM58377.2023.10356321.

Velazquez, A., Mathews, J.T., Lopes, R.R.F., Braun, T., Free-Nelson, F., 2023b. Toward autonomous cyber defense for protected core networking, in: International Conference on Military Communication and Information Systems (ICMCIS), Skopje, N. Macedonia. pp. 1–8.

Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Liu, Q., Liu, T., Sui, Z., 2023. Large Language Models are not Fair Evaluators, in: arXiv, pp. 1–11. doi:10.48550/arXiv.2305.17926.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2024. Chain-of-thought prompting elicits reasoning in large language models, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA. pp. 1–14. doi:10.5555/3600270.3602070.

Yang, S., Nachum, O., Du, Y., Wei, J., Abbeel, P., Schuurmans, D., 2023. Foundation Models for Decision Making: Problems, Methods, and Opportunities, pp. 1–32. doi:10.48550/arXiv.2303.04129.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv , 1–33doi:10.48550/arXiv.2210.03629.

Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.Y., Wen, J.R., 2023. A Survey of Large Language Models, in: arXiv, pp. 1–140. doi:10.48550/arXiv.2303.18223.

Zhu, W., Yu, C., Zhang, Q., 2023. Causal Deep Reinforcement Learning Using Observational Data, in: Elkind, E. (Ed.), Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23, International Joint Conferences on Artificial Intelligence Organization. pp. 4711–4719. doi:10.24963/ijcai.2023/524.

Johannes F. Loevenich is a scientist at Thales Deutschland in Ditzingen, Germany. Sitting with the Secure Communications & Information Systems (SIX), he dedicates his expertise to complex problems in Computer Networks and Distributed Systems with a particular interest in the development of robust and secure tactical systems using artificial intelligence. Before he was working as a scientist in the Communication Systems Department (KOM) at Fraunhofer FKIE in Bonn, Germany. His main challenge was to develop an intelligent system that is capable of learning the performance bounds of tactical systems from scratch. He received a MSc. Computer Science and another BSc. in Mathematics from Rheinische Friedrich-Wilhelms-Universität Bonn. Currently, he is pursuing a PhD in Computer Science/Mathematics in the Distributed Systems Department at the University of Osnabrück. His research interests include Computer Networks, Distributed Systems, Data Science, Optimization Theory, Artificial Intelligence, and Cyber Security.

Erik Adler is a student research assistant at Thales Deutschland in Ditzingen, Germany. He is currently pursuing a M.Sc. in Informatics at the Karlsruhe Institute of Technology. Moreover, he did a study abroad at the University of Economics Vienna out of curiosity for entrepreneurship. Prior to working at Thales Deutschland, he was working as a student research assistant at the Cognitive Distribution Systems Department at Fraunhofer IAO in Heilbronn. His research interests include intelligent autonomous systems, especially in the form of Large Language Models, in various domains like Computer Networks and Cyber Security.

Tobias Hürten is a student researcher at Thales Deutschland in Ditzingen, Germany. He currently pursues a Master's degree in Computer Science at the University of Bonn, focusing research on machine learning applications in the cyber security domain. Prior to joining Thales, he worked in the Communications Systems Department at Fraunhofer FKIE in Bonn, Germany, where he dedicated his research to developing secure tactical systems utilizing artificial intelligence and probing their robustness against adversarial attack vectors.

Roberto Rigolin F. Lopes is a scientist at Thales Deutschland in Ditzingen, Germany. Working with the Secure Communications & Information Systems (SIX) team, he has been attacking problems in Computer Networks and Distributed Systems with a particular interest in the performance bounds of tactical systems over ever-changing communication scenarios, and the design and development of autonomous cyber defense agents. His education includes B.Sc., M.Sc., and Ph.D. degrees in Computer Science from three universities in Brazil (UFMT, UFSCar, and USP). During his Ph.D., he also visited an university in the Netherlands (UTwente) and another in Canada (UOttawa). After his Ph.D., he got a postdoctoral scholarship from the European Research Consortium for Informatics and Mathematics (ERCIM) to join an university in Norway (NTNU). His academic life triggered interesting life experiences, but he has been rebuilding his own education following curiosity freely by reading books on Physics, Mathematics, and Philosophy.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5076836